

C++ Notes

Auto – Déclaration de type

Adam HAMOUCHE

May 2026

1. Définition

`auto` demande au compilateur de déduire le type d'une variable depuis son expression d'initialisation. C'est une option, pas une obligation — à utiliser là où ça apporte de la clarté ou de la sécurité.

2. Problèmes résolus

- Types longs ou verbeux à écrire (`std::unordered_map<...>::iterator`).
- Types uniquement connus du compilateur (lambdas, types internes).
- Conversions implicites silencieuses induites par une spécification de type incorrecte.
- Oubli de `const/&` dans une déclaration explicite.
- Verbose de `std::function` pour stocker un callable.

3. Fonctionnement

- Le type est déduit à la compilation depuis l'expression à droite du `=`.
- `auto` **abandonne** `const` et `&` par défaut — il faut les écrire explicitement : `const auto&`.
- Dans un template générique, `auto` permet d'être indépendant du type exact.
- Un paramètre de lambda est forcément `auto` (le type n'est pas connu à l'écriture).

4. Exemples

```
// Iterateur : auto evite la verbosit
std::unordered_map<std::string, Mesh*> meshes;
auto it = meshes.find("player"); // vs std::unordered_map<...>::iterator

// Lambda : le type n'existe qu'au niveau du compilateur
auto onHit = [&](Entity e) { applyDamage(e); };
// std::function<void(Entity)> onHit = ... => fonctionnel mais +lent, +
// lourd
```

```
// Conversion implicite silencieuse avec type explicite
std::vector<int> v;
unsigned sz = v.size();           // DANGER : unsigned != size_type sur
    certaines plateformes
auto      sz = v.size();           // OK : type exact = std::vector<int>::
    size_type

// auto oublie const et ref
const std::string& getName();
auto      name = getName(); // copie ! const et & ignores
const auto& name = getName(); // correct
```

5. Pièges classiques

- **Proxy types invisibles** : `auto x = vec.bool[i]` déduit `std::vector<bool>::reference`, pas `bool`. Dangling pointer si le vecteur était temporaire → comportement indéfini.
- **Expression templates** (Eigen, etc.) : `auto` capture l'expression lazy, pas le résultat calculé.
- **const/& silencieusement supprimés** : toujours qualifier `const auto&` si nécessaire.
- **Division entière** : `auto r = 1/3` → `int 0`, pas `float`.

Solution : explicitly typed initializer idiom

Quand `auto` déduit un proxy ou un type indésiré, forcer la conversion avec `static_cast` :

```
// Proxy type => cast explicite
auto highPriority = static_cast<bool>(features(w)[5]);

// Troncature voulue et documentee
auto index = static_cast<int>(ratio * vec.size());

// Perte de precision voulue et documentee
auto ep = static_cast<float>(calcEpsilon());
```

6. Coût et performance

- `auto` est zéro-cost : résolu entièrement à la compilation.
- Lambda stocké en `auto < std::function` : pas d'allocation heap, pas d'indirection virtuelle.
- Facilite le refactoring : changer le type retourné d'une fonction ne force pas à mettre à jour chaque déclaration.

7. Quand utiliser / éviter

7.1 Utiliser

- Itérateurs et types STL verbeux.
- Stockage de lambdas (toujours préférable à `std::function`).
- Templates génériques où le type exact ne doit pas être supposé.
- Type retourné par `make_unique` / `make_shared`.

7.2 Éviter

- Résultats de `operator[]` sur `std::vector<bool>` ou `std::bitset` → proxy invisible.
- Expression templates (Eigen, bibliothèques maths) sans `static_cast`.
- Quand l'ownership ou la sémantique `const`/ref est critique pour le lecteur.
- Quand le type n'est pas immédiatement évident depuis le contexte (lisibilité codebase engine multi-dev).